

API Contract

Base URLs

Local development:

```
http://127.0.0.1:8000
```

Public deployment:

```
https://kfc.theburkenator.com
```

Version prefix:

```
/api/v1
```

Swagger/OpenAPI docs:

```
https://kfc.theburkenator.com/docs
```

Authentication

Protected endpoints require:

```
Authorization: Bearer <access_token>
```

Access tokens are signed JWTs with required claims:

- `sub`: user UUID
- `role`: user role
- `jti`: token identifier
- `iat`: issued-at timestamp
- `exp`: expiry timestamp
- `type`: must be `access`

Refresh tokens are opaque random tokens. The server stores only `hmac_sha256:<digest>` refresh-token hashes in `refresh_sessions`.

Error Pattern

FastAPI/Pydantic validation errors use status 422 with a `detail` list. The app sanitizes validation errors so submitted secret values are not echoed.

Common safe errors:

Status	Example detail	Meaning
400	Message could not be processed	Valid JSON shape but invalid message/prekey/device workflow

Status	Example detail	Meaning
401	Invalid credentials	Login failed
401	Invalid authentication credentials	Missing, malformed, expired, wrong-signature, wrong-type, inactive-user, or nonexistent-user Bearer token
401	Invalid refresh token	Refresh failed
404	Message not found	Missing message or object not visible to current user
404	Target device not found	Missing/inactive target user or device for prekey bundle
409	Username or email is unavailable	Duplicate registration
409	One-time prekey already exists	Duplicate prekey for same user/device/prekey ID
404	Anchor not found	Missing blockchain anchor or anchor linked to a message hidden from current user
429	Too many requests	Rate limit exceeded

Auth Endpoints

POST /api/v1/auth/register

Registers a user. Does not issue login tokens.

Request:

```
{
  "username": "alice",
  "email": "alice@example.com",
  "password": "correct-horse-battery-staple"
}
```

Validation:

- username: 3-50 chars, letters/numbers/underscore/dot/hyphen only
- email: valid email address
- password: 12-128 chars
- extra fields rejected

Response 201:

```
{
  "id": "000000000-0000-0000-0000-000000000000",
  "username": "alice",
  "email": "alice@example.com",
  "role": "user",
}
```

```
"is_active": true,  
"created_at": "2026-06-02T12:00:00Z",  
"updated_at": "2026-06-02T12:00:00Z"  
}
```

No password_hash, access token, or refresh token is returned.

POST /api/v1/auth/login

Authenticates username or email and returns tokens.

Request:

```
{  
  "username_or_email": "alice",  
  "password": "correct-horse-battery-staple"  
}
```

Response 200:

```
{  
  "access_token": "<jwt>",  
  "refresh_token": "<opaque-refresh-token>",  
  "token_type": "bearer",  
  "expires_in": 900  
}
```

POST /api/v1/auth/refresh

Rotates a refresh token and returns a new access token and refresh token.

Request:

```
{  
  "refresh_token": "<opaque-refresh-token>"  
}
```

The old refresh token is revoked and cannot be reused.

POST /api/v1/auth/logout

Revokes the submitted refresh token if active. Returns success even when the token is unknown so validity is not disclosed.

Request:

```
{  
  "refresh_token": "<opaque-refresh-token>"  
}
```

Response:

```
{  
  "success": true,  
  "message": "Logged out"  
}
```

GET /api/v1/auth/me

Returns the authenticated user's safe profile.

User Discovery

GET /api/v1/users/by-username/{username}

Requires authentication. Returns a public-safe user/device summary for direct messaging setup.

Response:

```
{
  "id": "000000000-0000-0000-0000-000000000000",
  "username": "alice",
  "devices": [
    {
      "device_id": 1,
      "is_active": true
    }
  ]
}
```

Does not expose email, password hash, refresh-token hashes, public key material, or private key fields.

Key Endpoints

PUT /api/v1/keys/devices/{device_id}

Uploads or updates public key material for the current user's device.

Request:

```
{
  "device_id": 1,
  "registration_id": 1001,
  "identity_key_public_b64": "a2V5LW1hdGVyaWFs",
  "identity_signing_public_b64": "a2V5LW1hdGVyaWFs",
  "signed_prekey_id": 2001,
  "signed_prekey_public_b64": "a2V5LW1hdGVyaWFs",
  "signed_prekey_signature_b64": "a2V5LW1hdGVyaWFs"
}
```

Rules:

- Path `device_id` must match request `device_id`.
- Key/signature fields must be valid non-empty standard base64.
- Private key, ratchet state, and session state fields are rejected.

POST /api/v1/keys/devices/{device_id}/one-time-prekeys

Uploads public one-time prekeys for the current user's existing active device.

Request:

```
{
```

```

"prekeys": [
  {
    "device_id": 1,
    "prekey_id": 10,
    "prekey_public_b64": "a2V5LW1hdGVyaWFs"
  }
]
}

```

Rules:

- Batch size: 1-100.
- Duplicate (device_id, prekey_id) pairs in one batch are rejected.
- Duplicate database prekeys for the same user/device/prekey ID return 409.
- Upload requires the current user to own an active device row.

GET

/api/v1/keys/users/{user_id}/devices/{device_id}/prekey-bundle

Returns public prekey bundle material for a target active user/device. Requires authentication.

Response uses crypto-package-compatible camelCase aliases:

```

{
  "registrationId": 1001,
  "deviceId": 1,
  "identityKey": "a2V5LW1hdGVyaWFs",
  "identitySigningKey": "a2V5LW1hdGVyaWFs",
  "signedPreKeyId": 2001,
  "signedPreKey": "a2V5LW1hdGVyaWFs",
  "signedPreKeySignature": "a2V5LW1hdGVyaWFs",
  "oneTimePreKeyId": 10,
  "oneTimePreKey": "a2V5LW1hdGVyaWFs"
}

```

If an unused one-time prekey exists, the route marks one prekey used and returns it. If none exists, the one-time fields are null.

Message Endpoints

All message routes require authentication.

POST /api/v1/messages

Stores a direct 1:1 encrypted relay message.

Request:

```

{
  "sender_device_id": 1,
  "recipient_user_id": "00000000-0000-0000-0000-000000000000",
  "recipient_device_id": 1,
  "wire_payload_json": "{\"format\":\"libsignal-v1\",\"type\":3,\"bodyB64\":\"b3JpZ2luYWw=\",\"registrationId\":\"12345\"}",
  "consumed_one_time_prekey_id": 10
}

```

```
}
```

Rules:

- Sender user ID is always `current_user.id`.
- Sender and recipient devices must exist and be active.
- Recipient user must exist and be active.
- `consumed_one_time_prekey_id`, when present, must refer to a recipient/device prekey already marked used.
- `wire_payload_json` maximum size is 64 KiB.
- A pending blockchain anchor is created automatically in the same transaction as the message.

`wire_payload_json` must contain a JSON object matching:

```
{
  "format": "libsignal-v1",
  "type": 1,
  "bodyB64": "base64-ciphertext",
  "registrationId": 12345
}
```

Allowed message types are 1 and 3. `registrationId` is optional but must be a positive integer if present.

GET /api/v1/messages/received

Lists messages visible to the current user as recipient. Supports query parameters:

- `limit`: default 50, min 1, max 100
- `offset`: default 0, min 0

Revoked, recipient-deleted, and globally deleted messages are excluded.

GET /api/v1/messages/sent

Lists messages visible to the current user as sender. Supports `limit` and `offset`.

Sender-deleted and globally deleted messages are excluded.

GET /api/v1/messages/{message_id}

Returns a message only if the current user is the sender or visible recipient.

GET /api/v1/messages/{message_id}/anchor

Returns the latest blockchain anchor for a message only if the current user can access that message.

Response:

```
{
  "id": "000000000-0000-0000-0000-000000000000",
  "message_id": "000000000-0000-0000-0000-000000000000",
}
```

```

    "batch_id": null,
    "record_id":
"0x0000000000000000000000000000000000000000000000000000000000000000",
    "digest":
"0x1111111111111111111111111111111111111111111111111111111111111111",
    "merkle_root": null,
    "transaction_hash": null,
    "contract_address": null,
    "chain": "sepolia",
    "status": "pending",
    "created_at": "2026-06-02T12:00:00Z",
    "anchored_at": null
}

```

POST /api/v1/messages/{message_id}/forward

Requires access to the original message and stores a new encrypted payload for a new recipient.

A forwarded message is a new encrypted message row and receives its own pending blockchain anchor. The backend does not decrypt the original message or copy plaintext.

The backend stores `forwarded_from_message_id` on the new row using the `message_id` path parameter after access to the original message has been verified. Clients must not submit `forwarded_from_message_id` in request bodies.

POST /api/v1/messages/{message_id}/revoke

Sender-only. Sets `access_revoked_at`, removing recipient access.

DELETE /api/v1/messages/{message_id}

Hides a message from the current user only:

- Sender delete sets `sender_deleted_at`.
- Recipient delete sets `recipient_deleted_at`.

It does not immediately hard-delete the other party's copy.

Blockchain Anchor Endpoints

All blockchain routes require authentication. They store and return integrity metadata only. FastAPI request handlers do not submit Ethereum transactions; the backend blockchain worker submits pending anchors asynchronously.

POST /api/v1/blockchain/anchors

Creates or reuses a pending anchor for an accessible message. This route is mainly a manual retry/demo path because message send and forward already create pending anchors automatically.

Request:

```

{
  "message_id": "00000000-0000-0000-0000-000000000000"
}

```

Response:

- 201 Created when a new pending anchor is created
- 200 OK when an existing anchor is reused

The response shape is `BlockchainAnchorResponse`.

GET /api/v1/blockchain/anchors/{anchor_id}

Returns anchor status only if the current user can access the linked message.

Status values:

- pending
- confirmed
- failed

POST /api/v1/blockchain/verify

Checks supplied digest/root metadata against confirmed backend anchor records.

Request:

```
{
  "digest":
  "0x1111111111111111111111111111111111111111111111111111111111111111",
  "record_id":
  "0x0000000000000000000000000000000000000000000000000000000000000000",
  "merkle_root": null,
  "transaction_hash": null,
  "chain": "sepolia"
}
```

Response:

```
{
  "valid": true,
  "chain": "sepolia",
  "status": "confirmed",
  "anchor_id": "00000000-0000-0000-0000-000000000000",
  "message_id": "00000000-0000-0000-0000-000000000000",
  "batch_id": null,
  "record_id":
  "0x0000000000000000000000000000000000000000000000000000000000000000",
  "digest":
  "0x1111111111111111111111111111111111111111111111111111111111111111",
  "merkle_root": null,
  "transaction_hash":
  "0x2222222222222222222222222222222222222222222222222222222222222222",
  "contract_address": "0x3333333333333333333333333333333333333333333333333333333333333333",
  "anchored_at": "2026-06-02T12:30:00Z",
  "verified_at": "2026-06-02T12:31:00Z"
}
```

This endpoint does not contact Sepolia live. The backend blockchain worker writes pending anchors to the Solidity contract and updates the database with confirmation metadata.

Explicitly Rejected Input

Request schemas reject unexpected fields including:

- `sender_user_id`
- `conversation_id`
- group or client-supplied conversation fields
- `body`
- `content`
- `plaintext`
- private key fields
- ratchet/session/root/chain key fields inside `wire_payload_json`
- raw refresh-token storage fields
- password hash fields

The backend stores encrypted relay payloads, public keys, token-session hashes, blockchain integrity metadata, and audit metadata only.